



REPORTE DE PRÁCTICA NO. 6

NOMBRE DE LA PRÁCTICA: Análisis sintáctico de expresiones aritméticas

ALUMNOS: Garcia Colmenares Andres
Arana Juarez Raul
Flores Guzman Oliver Alain

Dr. Eduardo Cornejo-Velázquez

1. Introducción

Una Gramática Formal definida sobre un alfabeto Σ es una tupla de la forma:

$$G = \Sigma_T, \Sigma_N, S, P$$

donde:

- Σ_T es un alfabeto de símbolos terminales
- Σ_N es un alfabeto de símbolos no terminales
- S es el símbolo inicial de la gramática
- P es un conjunto de producciones gramaticales

Además, se cumple:

- $S \in \Sigma_N$
- $\Sigma_T \cap \Sigma_N = \emptyset$
- $\Sigma = \Sigma_T \cup \Sigma_N$

La gramática formal G permite generar un lenguaje $L = \{x \in \Sigma_T^* \mid S \rightarrow x\}$. Por lo tanto, las palabras del lenguaje estarán formadas por cadenas de símbolos terminales generadas a partir del símbolo inicial de la gramática utilizando las producciones que la definen.

Una expresión regular es una notación normalizada para representar lenguajes regulares, es decir, lenguajes generados por gramáticas regulares. Las expresiones regulares permiten describir con exactitud y sencillez cualquier lenguaje regular.

El analizador sintáctico es la parte del compilador que se encarga de analizar el código de entrada en base a una gramática dada y, en caso de que el programa de entrada sea válido, construye el árbol sintáctico que lo reconoce.

En teoría, se supone que la salida del analizador sintáctico es alguna representación del árbol sintáctico que reconoce la secuencia de tokens suministrada por el analizador léxico, pero en la práctica, el analizador sintáctico también identifica errores cuando se producen y se comunica con la tabla de símbolos para almacenar información de los procesos realizados.

Un analizador sintáctico es un programa que analiza códigos para determinar su estructura sintáctica.

Ejemplo de análisis sintáctico aritmético: La estructura sintáctica de la cadena $2 * 3 + 4$ es el siguiente árbol:

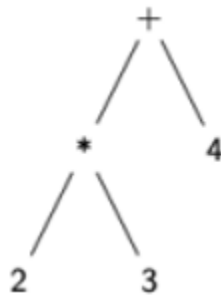


Figure 1: Árbol sintactico

El análisis sintáctico forma parte del preprocesamiento en la mayoría de las aplicaciones reales. Consideramos expresiones aritméticas:

construidas con números, operaciones (+ y *) y paréntesis.

y * asocian por la derecha.

tiene más prioridad que +.

2. Marco teórico

Lenguajes y Gramáticas Formales

Un lenguaje formal es un conjunto de cadenas de símbolos construidas sobre un alfabeto finito. Para describir la sintaxis de un lenguaje de programación (o de un subconjunto como las expresiones aritméticas) se utilizan gramáticas formales.

Una gramática formal se define como una tupla:

$$G = (\Sigma_T, \Sigma_N, S, P)$$

Donde:

- Σ_T : conjunto finito de símbolos terminales (tokens, como números, operadores, paréntesis). No pueden ser reemplazados.
- Σ_N : conjunto finito de símbolos no terminales (categorías sintácticas, como expresión, término, factor). Pueden ser reemplazados por producciones
- $S \in \Sigma$: símbolo inicial de la gramática.
- P : conjunto de producciones de la forma $A \rightarrow \alpha$ donde $A \in \Sigma_N$ y $\alpha \in (\Sigma_T \cup \Sigma_N)^*$

El lenguaje generado por G es:

$$L(G) = \{ x \in \Sigma_T^* \mid S \rightarrow^* x \}$$

Es decir, todas las cadenas de terminales que se pueden derivar desde S .

Clasificación de Gramáticas

- Tipo 3 (Gramáticas Regulares): Se utilizan para definir patrones léxicos (identificadores, números, etc.). Pueden ser representadas por expresiones regulares o autómatas finitos.
- Tipo 2 (Gramáticas Libres de Contexto - GLC): Se utilizan para definir la sintaxis de construcciones anidadas como expresiones aritméticas, estructuras de control, etc. Son la base de los analizadores sintácticos como Yacc.

En esta práctica:

- El analizador léxico (Lex) reconoce tokens mediante expresiones regulares (Tipo 3).
- El analizador sintáctico (Yacc) verifica la estructura de las expresiones mediante una gramática libre de contexto (Tipo 2).

Analizador Léxico (Lex/Flex)

El analizador léxico (lexer) convierte una cadena de caracteres en una secuencia de tokens. Cada token es un par (tipo, valor).

Tokens requeridos en la práctica:

- Números enteros $([0-9]^+)$
- Números decimales $([0-9]^+ \cdot [0-9]^+)$
- Operadores aritméticos de Python: $+$, $-$, $*$, $/$ Operadores relacionales (para condicionales): $<$, $<=$, $>$, $>=$, $==$, $!=$
- Identificadores (variables): $[a-zA-Z][a-zA-Z0-9]^*$

- Paréntesis: (,)

Autómata finito: Para reconocer estos patrones se puede diseñar un autómata finito determinista (AFD) o no determinista (AFN), que Lex implementa automáticamente a partir de las expresiones regulares.

Analizador Sintáctico (Yacc/Bison)

El analizador sintáctico (parser) recibe los tokens del lexer y verifica si forman una estructura válida según una gramática libre de contexto. Si es válida, puede construir un árbol sintáctico (parse tree) o un AST (Abstract Syntax Tree).

Consideraciones:

- Prioridad de operadores: * y / tienen mayor prioridad que + y -. Esto se logra colocando los operadores de mayor prioridad en reglas más profundas (término $\rightarrow$ factor).
- Asociatividad: Por defecto, Yacc asocia por izquierda. Para operadores como + y * es correcto. Si se requiere asociatividad por derecha (como en potencias), se usa

Derivaciones y Árbol Sintáctico

Una derivación es una secuencia de reemplazos de no terminales usando las producciones.

- Derivación por la izquierda: Se reemplaza siempre el no terminal más a la izquierda.
- Derivación por la derecha: Se reemplaza siempre el no terminal más a la derecha.

El árbol sintáctico representa gráficamente la estructura jerárquica de una expresión. Cada nodo interno es un no terminal (o un operador), y las hojas son terminales.

Ejemplo: Para la expresión $2 * 3 + 4$ con la gramática anterior, la derivación por la izquierda genera un árbol que refleja que * tiene mayor prioridad, evaluándose primero $2*3$ y luego sumando 4.

Manejo de Errores Sintácticos

Yacc permite detectar errores sintácticos cuando la secuencia de tokens no se puede derivar desde el símbolo inicial. Los errores comunes incluyen:

- Operadores consecutivos sin operandos (ej: $2 + * 3$)
- Paréntesis desbalanceados
- Faltan operandos (ej: $2 +$)
- Identificadores mal formados (aunque eso lo detecta el lexer)

El uso de una gramática bien definida permite identificar dónde y por qué ocurre el error.

3. Herramientas empleadas

Para esta practiac se usaron dos herramientas principales.

1. Lex (o Flex): Genera un analizador léxico en C a partir de un archivo con definiciones de expresiones regulares y acciones (retornar tokens).
2. Yacc (o Bison): Genera un analizador sintáctico en C a partir de una gramática libre de contexto con acciones asociadas (validación, construcción de AST, mensajes de error).

Ambas herramientas siguen el modelo de análisis LALR(1) (Look-Ahead Left-to-Right, con 1 token de anticipación), muy eficiente para la mayoría de los lenguajes de programación.

4. Desarrollo

Base

Para esta practica se partió de un código proporcionado por el docente, el cual sirvió como base para desarrollar el nuevo proyecto funcionando bajo el mismo principio.

Desarrollo del programa en Lex para reconocer los tokens:

- Números enteros.
- Números decimales.
- Símbolos que representan las operaciones de suma, resta, multiplicación y división.
- Símbolos para construir las expresiones condicionales (por ejemplo, comparaciones).
- Identificadores (Nombres de variables)
- Paréntesis.

Código en Lex:

```
%option noyywrap
%{
#include <stdio.h>
#include "y.tab.h"
}%

%%
[0-9]+                { return ENTERO; }
[0-9]+\.[0-9]+        { return DECIMAL; }
[+\-*\/]              { return SIMBOLO; }
(==|!=|<=|>=|<|>)    { return COMPARACION; }
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }
"("                   { return '('; }
")"                   { return ')'; }

[ \t]                 { }           // ignorar espacios
\n                    { return EOL; }

%%
```

Figure 2: Código Lex

Desarrollo del programa en Yacc para:

- Verificar que las operaciones aritméticas planteadas cumplan las reglas definidas.
- Agregar acciones que confirmen si la expresión es válida o no.

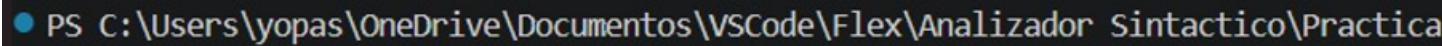
```
1 parser.y
2 %{
3 #include <stdio.h>
4 int yylex();
5 int yyerror(const char *s) {
6     printf("Expresion NO valida\n");
7     return 0;
8 }
9 %}
10
11 %token ENTERO DECIMAL SIMBOLO COMPARACION ID EOL
12 %left SIMBOLO
13 %left COMPARACION
14 %start input
15
16 %%
17
18 input:
19     input expr EOL { printf("Expresion valida\n"); }
20     | /* vacío */
21 ;
22
23 expr:
24     expr SIMBOLO expr
25     | expr COMPARACION expr
26     | '(' expr ')'
27     | ENTERO
28     | DECIMAL
29     | ID
30 ;
31
32 %%
33
34 int main() {
35     yyparse();
36     return 0;
37 }
```

Figure 3: Código Yacc

Ejecutar los programas en Lex y Yacc

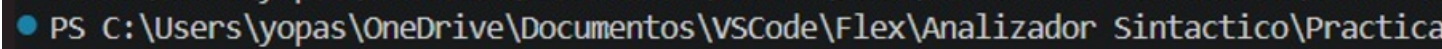
Para generar el programa ejecutable es necesario primero hacer ciertos pasos con los programas generados anteriormente ya que se requiere parte de ambos para que el analizador sintactico funcione.

1. Paso 1: Como primera accion debemos ubicarnos en la carpeta exacta en la que se encuentran nuestros archivos.
2. Posteriormente compilamos ambos programas para obtener los componentes que se requieren para la generación del programa final.



```
● PS C:\Users\yopas\OneDrive\Documentos\VSCode\Flex\Analizador Sintactico\Practica
```

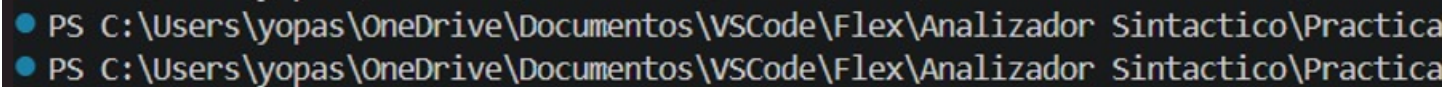
Figure 4: Compilación yacc



```
● PS C:\Users\yopas\OneDrive\Documentos\VSCode\Flex\Analizador Sintactico\Practica
```

Figure 5: Compilación lex

3. Por ultimo vamos a usar los archivos generados a partir de los codigos anteriores para generar el ejecutable.



```
● PS C:\Users\yopas\OneDrive\Documentos\VSCode\Flex\Analizador Sintactico\Practica
● PS C:\Users\yopas\OneDrive\Documentos\VSCode\Flex\Analizador Sintactico\Practica
```

Figure 6: Generación del archivo ejecutable

Pruebas y resultados

Nuestro analizador sintactico se basó en la entrada desde la consola como se muestra a continuacion con los casos de aceptación y rechazo.

```
● PS C:\Users\yopas\OneDrive\Documentos\VSCode\Flex\Analizador Sintactico\Pruebas> (2 + 2) / 2  
Expresion valida  
(2 + 2) / 6  
Expresion valida  
(2 + 2) * 6  
Expresion valida  
(2 + 2) * 6.4  
Expresion valida  
(2 + 2.1213) * 6.4
```

Figure 7: Casos de aceptación

```
● PS C:\Users\yopas\OneDrive\Documentos\VSCode\Flex\Analizador Sintactico\Pruebas> 2 + 22  
Expresion valida  
2 + 222  
Expresion valida  
2 +++ 2 + 2++  
Expresion NO valida
```

Figure 8: Casos de rechazo

5. Cuestionario

1. **¿Qué tipo de errores fueron detectados en los ejemplos incorrectos y cómo influyeron las gramáticas definidas para lograr su identificación?**

Se generaron errores de definicion y de ambigüedad, las gramaticas definidas ayudaron a corregir estos errores, especialmente el de ambigüedad.

2. **¿Cómo contribuye el uso de derivaciones por la izquierda o por la derecha en la construcción de un árbol sintáctico para validar las operaciones y las prioridades de las operaciones?**

Las derivaciones hacen que la ambigüedad se rompa, porque así se tiene el paso definido para cada operacion del analizador sintactico

3. **¿Qué ajustes serían necesarios en la gramática para permitir el reconocimiento de estructuras condicionales más complejas, como $(a + b)^2$, raíz cuadrada, así como operaciones trigonométricas?**

Definir cada token correctamente principalmente, dar a conocer las variables por izquierda así como las de derecha segun su jerarquia en el orden de operacion, etc

6. Conclusiones

A lo largo del desarrollo de la practica pusimos en practica todos los conocimientos que hemos adquirido a lo largo del semestre ya que necesitamos del analizador lexico para poner en practica el analisis sintactico. Parte del aprendizaje practico vino cuando llego el momento de compilar, ya que se tuvieron algunas dificultades para llevar a cabo la generacion del archivo ejecutable.

Logramos comprender el funcionamiento de ambos analisis y como se relacionan entre si, ademas de que comprobamos de manera practica lo que necesitan el uno del otro para poder funcionar de manera correcta.

Referencias Bibliográficas

References

- [1] Giró, J., Vázquez, J., Meloni, B., Constable, L (**2015**). Lenguajes formales y teoría de autómatas. Editorial Alfaomega. Argentina.
- [2] Gutú, O. (**2013**). Primer curso en teoría de autómatas y lenguajes formales. Pearson Educación. México.
- [3] Hopcroft, J. E., Motwani, R. & Ullman, J. D. (**2007**). Introducción a la teoría de autómatas...
- [4] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (**2008**). Compiladores: principios, técnicas y herramientas (2.^a ed.). Pearson Educación.